

แบบทดสอบท้ายแล็บ

Custom ViewResolver ใน Spring Boot + Thymeleaf

วิชา CP353002 Principles of Software Design

ชื่อ-นามสกุล: นายดริณภพ สุริเตอร์ รหัสนักศึกษา: 673380402-4

คำชี้แจง: ส่วนที่ 1 ปรนัย เลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว ส่วนที่ 2 อัตนัย ตอบเป็นข้อความอธิบาย ส่วนที่ 3 ปฏิบัติ ให้ลงมือแก้ไขโค้ดจริงในโปรเจกต์ของตัวเอง แล้ว capture หน้าจอผลลัพธ์แปะในกล่องที่เตรียมไว้

ส่วนที่ 1 — ปรนัย (Multiple Choice)

1. ในสถาปัตยกรรม MVC ส่วนใดที่ทำหน้าที่แปลง "ชื่อ view" ซึ่งตรงกระให้กลายเป็น path ของไฟล์ view จริง?

(2 คะแนน)

ก. Model

ข. Controller

ค. ViewResolver

ง. DispatcherServlet

2. เมื่อ Controller เขียน return "home"; ค่า "home" ที่ถูกส่งกลับคืออะไร? (2 คะแนน)

ก. Path ของไฟล์ HTML บนเครื่อง server

ข. ชื่อ view ซึ่งตรงกระ (logical view name)

ค. ชื่อ bean ของ ViewResolver

ง. ชื่อ method ของ Controller

3. component ไດใน Spring MVC เป็นผู้เรียกใช้ ViewResolver โดยอัตโนมัติ หลังจาก Controller คืนค่าชื่อ view? (2 คะแนน)

ก. HandlerMapping

ข. DispatcherServlet

ค. SpringTemplateEngine

ง. HomeController

4. ใน ThymeleafConfig.java เมธอด resolver.setPrefix("classpath:/custom-templates/") ทำหน้าที่อะไร? (2 คะแนน)

ก. กำหนด URL ที่ Controller จะรับ request

ข. กำหนดไฟล์เดอร์ต้นทางที่ ViewResolver จะค้นหาไฟล์ template

ค. กำหนดชื่อ database connection

ง. กำหนด port ที่แอปพลิเคชันรัน

5. ถ้ามีการลงทะเบียน ViewResolver มากกว่าหนึ่งตัวในบริบท (context) เดียวกัน Spring จะเลือกใช้ตัวไหนก่อน? (2 คะแนน)

ก. ตัวที่ประกาศเป็นตัวแรกในไฟล์เสมอ

ข. ตัวที่มีค่า order ต่ำกว่า (ถูกเรียกก่อน)

ค. สุ่มเลือก

ง. ตัวที่ตั้งชื่อ bean เรียงตามตัวอักษร

6. เพราะเหตุใด Controller ในตัวอย่างนี้จึงใช้ @Controller แทน @RestController? (2 คะแนน)

ก. เพราะ @RestController ใช้กับ Thymeleaf ไม่ได้เลย

ข. เพราะต้องการให้ค่าที่ return ถูกตีความเป็นชื่อ view ไม่ใช่ response body โดยตรง

ค. เพราะ @Controller ทำงานเร็วกว่า

ง. ไม่มีความแตกต่างกันเลย

ส่วนที่ 2 — อัดนัย (Short Answer / Essay)

1. ใครเป็นผู้เรียกใช้ Custom ViewResolver และเรียกเมื่อไร? จงอธิบายเป็นลำดับขั้นตอน ตั้งแต่ Browser ส่ง request จนได้ HTML response กลับ (5 คะแนน)

- Browser ส่ง HTTP Request ไปยัง Server
- DispatcherServlet รับ Request และส่งต่อให้ Controller
- Controller ประมวลผลแล้ว return "home" ซึ่งเป็น Logical View Name
- DispatcherServlet เรียกใช้ Custom ViewResolver เพื่อแปลง "home" ไปเป็น path ของไฟล์ Template เช่น classpath:/custom-templates/home.html
- View ถูก Render เป็น HTML แล้วส่ง HTML Response กลับไปยัง Browser

2. จงอธิบายว่าเหตุใดการแยก ViewResolver ออกจาก Controller จึงสอดคล้องกับหลักการ separation of concerns และ dependency inversion ใน Principles of Software Design (5 คะแนน)

การแยก ViewResolver ออกจาก Controller สอดคล้องกับ Separation of Concerns เพราะ Controller รับผิดชอบเพียงการจัดการ Request และกำหนดชื่อ View ส่วน ViewResolver รับผิดชอบค้นหาและแปลงชื่อ View เป็นไฟล์จริง ทำให้แต่ละส่วนมีหน้าที่ชัดเจนและแก้ไขได้แยกกัน

3. หากต้องการเพิ่ม view ใหม่ชื่อ "about" โดยยังใช้ custom ViewResolver ตัวเดิม ต้องสร้างหรือแก้ไขไฟล์ใดบ้าง? ระบุชื่อไฟล์และ path ให้ครบถ้วน (3 คะแนน)

เพิ่ม/แก้ HomeController.java

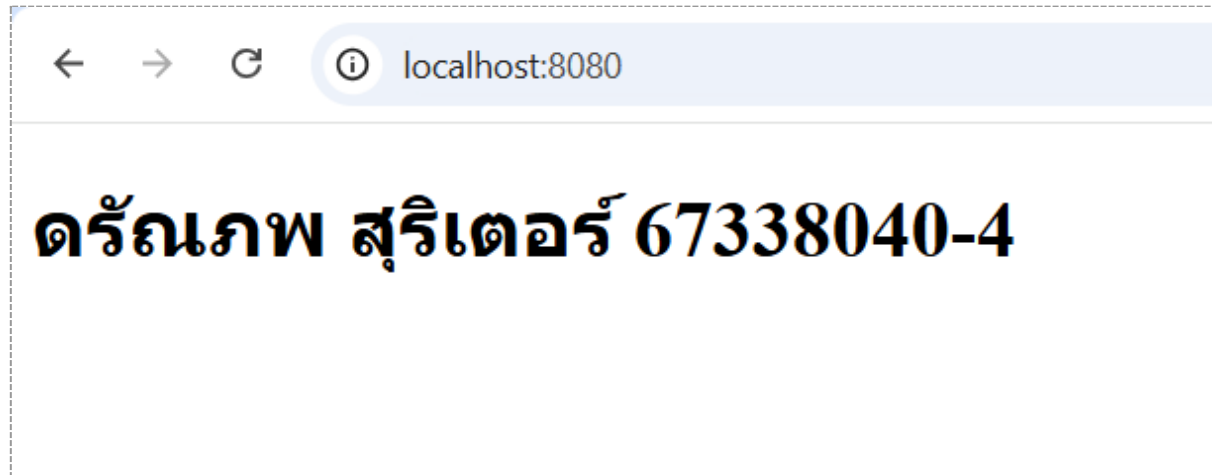
และสร้าง about.html ที่ src/main/resources/custom-templates/about.html

ส่วน ThymeleafConfig.java

ส่วนที่ 3 — ปฏิบัติ (แก้โค้ดจริง + แบนภาพหน้าจอ)

ทำตามคำสั่งแต่ละข้อในโปรเจกต์ของตัวเอง รันด้วย `mvn spring-boot:run` แล้ว *capture* หน้าจอผลลัพธ์แปะในกล่องเส้นประด้านล่างแต่ละข้อ

1. แก้ไขค่าตัวแปร `message` ใน `HomeController.java` ให้แสดงชื่อ-นามสกุลของตัวเองแทนข้อความเดิม แล้ว *capture* หน้าเว็บ `http://localhost:8080/` ที่แสดงชื่อของตัวเอง (3 คะแนน)



2. เพิ่มตัวแปรใหม่ชื่อ `studentId` ส่งจาก Controller ไปแสดงต่อจากชื่อในหน้า `home.html` (เช่น "สวัสดี ชื่อ (รหัส XXX)") แล้ว *capture* หน้าเว็บที่แสดงผลลัพธ์ (3 คะแนน)



3. เปลี่ยนค่า `resolver.setPrefix()` ใน `ThymeleafConfig.java` ให้ชี้ไปโฟลเดอร์ใหม่ชื่อ `my-templates/` แทน `custom-templates/` (ต้องย้ายไฟล์ `home.html` ไปด้วย) แล้ว *capture* ทั้งโค้ดที่แก้ และหน้าเว็บที่ยังทำงานปกติ (4 คะแนน)

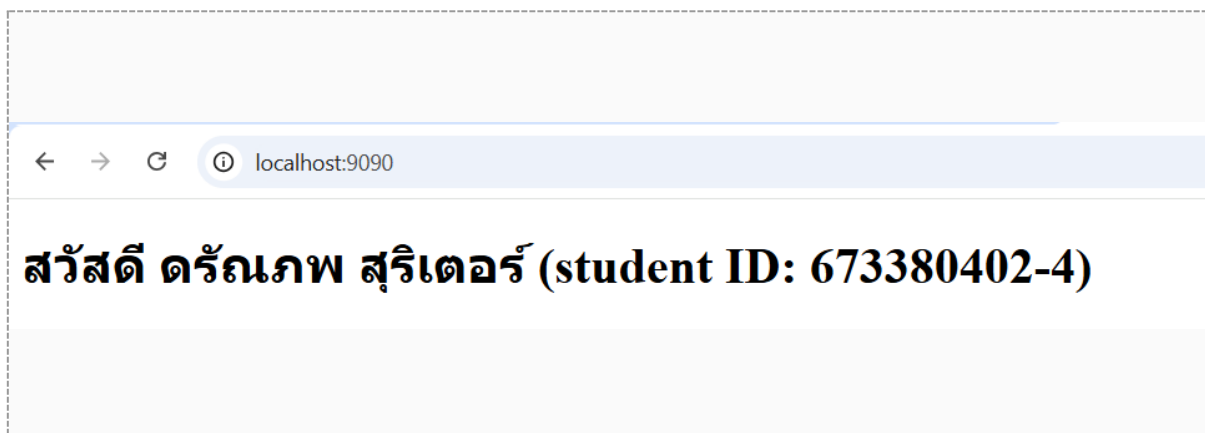


```
6 import org.thymeleaf.spring6.templateresolver.SpringResourceTemplateResolver;
7 import org.thymeleaf.spring6.view.ThymeleafViewResolver;
8
9 @Configuration
10 public class ThymeleafConfig {
11
12     @Bean
13     public SpringResourceTemplateResolver templateResolver() {
14         SpringResourceTemplateResolver resolver = new SpringResourceTemplateResolver();
15         resolver.setPrefix("classpath:/my-templates/"); // ไฟล์เตอร์ที่กำหนดเอง
16         resolver.setSuffix(".html");
17         resolver.setTemplateMode("HTML");
18         resolver.setCharacterEncoding("UTF-8");
19         resolver.setCacheable(false); // ปิด cache ระหว่าง dev
20         return resolver;
21     }
22
23     @Bean
24     public SpringTemplateEngine templateEngine(SpringResourceTemplateResolver templateResolver) {
25         SpringTemplateEngine engine = new SpringTemplateEngine();
26         engine.setTemplateResolver(templateResolver);
27         return engine;
28     }
29 }
```

localhost:8080

สวัสดี ดรณภพ สุริเตอร์ (student ID: 673380402-4)

4. เปลี่ยน server.port ใน application.properties เป็น 9090 แล้ว capture หน้าเว็บที่ URL เปลี่ยนเป็น <http://localhost:9090/> (3 คะแนน)



localhost:9090

สวัสดี ดรณภพ สุริเตอร์ (student ID: 673380402-4)

5. เพิ่ม view ใหม่ชื่อ "about" (เพิ่ม @GetMapping("/about") และไฟล์ about.html) ให้แสดงข้อความแนะนำตัวสั้นๆ แล้ว capture หน้าเว็บ /about ที่แสดงผลลัพธ์ (4 คะแนน)



localhost:9090/about

ข้อรณภพ สรเตอร